

Parametric IFC Design Studio

How requirements, zoning and BaaP rules become a viewable building

RNGD side-testing project

September 24, 2026

Contents

1	Summary	2
2	Background: the terms used in this report	2
3	System overview	3
4	Inputs in detail	3
4.1	Requirements (what the customer wants)	3
4.2	Zoning presets (the rules)	4
4.3	BaaP catalog (the building blocks)	4
5	How the layout is generated	4
5.1	Step 1: buildable envelope	5
5.2	Step 2: how many of each unit	5
5.3	Step 3: spreading the unit types evenly	5
5.4	Step 4: the shape of one wing	5
5.5	Step 5: storeys and heights	5
5.6	Step 6: packing units floor by floor	5
5.7	Step 7: accessible units	6
5.8	Step 8: parking	6
5.9	Step 9: choosing the number of wings	6
6	The ten compliance checks	6
7	How the IFC file is written	6
8	The viewer and the app	7

9	Worked example: the 120-unit baseline	7
10	Results	8
10.1	Reading the scenarios	9
11	Testing	10
12	Limitations and next steps	10
13	How to run	11

1 Summary

This test answers one question: *can we go from customer requirements, zoning rules and a catalog of standard building modules to a real 3D building file, using only free, open-source tools?* The answer is yes. A Python program reads the requirements, applies a zoning preset and the BaaP (Building-as-a-Product) module catalog, lays out a building, and writes it as a standard **IFC4** file using **IfcOpenShell**. The file is then re-read from disk and displayed in Chrome as an interactive 3D model, with buttons and sliders that regenerate the design instantly. Every design is scored against ten pass/fail checks so you can see why a design works or fails.

Three important honesty notes. (1) All zoning rules and BaaP module data are **synthetic samples**; they stand in for RNGD’s real data. (2) The layout is **planning-level**: it is a rule-based massing and unit layout, not a code-approved drawing set. (3) No Revit file (**.rvt**) is produced; that step was dropped from scope. The IFC opens in free BIM tools instead.

2 Background: the terms used in this report

IFC (Industry Foundation Classes)

The open, vendor-neutral file format for building models (ISO 16739). It stores real building objects (walls, slabs, storeys) with geometry and names, not just pictures. Most BIM software can open it.

IfcOpenShell

A free Python library that creates, reads and triangulates IFC files. We use it both to write the design and to read it back.

BaaP (Building-as-a-Product)

RNGD’s idea of building from a fixed catalog of standard modules (unit types, cores, corridors) instead of designing each building from scratch. Here the catalog is in **data/baap_catalog.json**.

Setback

The strip along each lot edge where building is not allowed. **Easement**: a strip reserved for utilities, also unbuildable.

Buildable envelope

The rectangle left after removing setbacks and easements. Everything must fit inside it.

FAR (floor area ratio)	Total building floor area divided by lot area. Limits how much building you can put on a lot.
Lot coverage	Building footprint area divided by lot area, as a percent.
Double-loaded corridor	A hallway with unit modules on both sides; the most efficient way to pack apartments.
Core	The block with stairs, elevator and shafts that serves the corridor. Wing: one long bar of corridor plus units. A building may have one to three wings.
Podium parking	A parking level built into the base of the building, instead of surface stalls, used when the lot is too small for surface parking.

3 System overview

The data flow is a straight line from requirements to a file you can open:

requirements+zoning preset+BaaP catalog → `design.py` → `ifc_writer.py` → `.ifc` → `viewer3d.py` → C

File	Job
<code>design.py</code>	Holds the zoning presets and parameters; computes the envelope, packs units, places driveway and parking, and runs the ten compliance checks. Pure Python, all dimensions in feet.
<code>ifc_writer.py</code>	Converts a layout to an IFC4 file (metres) and also reads an IFC back into triangle meshes.
<code>viewer3d.py</code>	Turns the meshes into an interactive Plotly 3D figure, colored by unit type.
<code>app.py</code>	The Streamlit browser app: buttons, sliders, tabs, downloads. Optionally uses the existing AI intake agent to parse a pasted customer request.
<code>generate.py</code>	Command-line tool that writes every scenario to <code>out/</code> as <code>.ifc</code> and a standalone <code>.html</code> viewer.
<code>test_design.py</code>	12 automated tests. <code>build_report.py</code> builds this document.

4 Inputs in detail

4.1 Requirements (what the customer wants)

These are fields of the `Params` object and the app's sidebar controls.

Input	Default	Meaning
Units	120	Total apartments requested (a hard requirement in the sample request).
Unit mix	20/45/35 %	Share of studio, 1-bedroom, 2-bedroom units.
Residential floors	4	Number of floors that contain apartments.
Parking type	surface	surface stalls on the ground, or podium parking under the building.
Wings	auto	0 lets the program try 1, 2 and 3 wings and pick the best; 1–3 forces a choice.
Lot	200×300 ft	Frontage × depth; street at the top; 20 ft utility easement on the east side.
Story heights	14 / 10.5 ft	Ground (or podium) level and typical upper level.

In the app, a free-text request can be pasted and the AI intake agent (Groq) extracts units, floors and mix into these controls automatically.

4.2 Zoning presets (the rules)

Three synthetic presets are included so you can see how the same building behaves under different rules. Setbacks are front/side/rear in feet.

Preset	Setbacks	FAR	Cov. %	Height	Stories	Parking/unit
Rivermont R-4 (sample)	15/10/20	2.5	60	65	5	1.25
Dense urban R-6 (sample)	10/5/15	3.5	75	85	7	0.75
Suburban R-2 (sample)	25/10/25	1.0	40	35	3	2.0

4.3 BaaP catalog (the building blocks)

Unit modules are fixed products; the design problem is choosing how many of each and where they go.

Module	Type	Width × depth (ft)	GSF	Notes
STUDIO_A	Studio	26 × 32	550	
ONEBR_A	1-bedroom	30 × 34	725	
TWOBR_A	2-bedroom	34 × 36	1020	
ACCESSIBLE_1BR	1-bedroom (ADA)	32 × 34	760	counted for the 2% rule
CORE_STD	Stairs + elevator core	24 along corridor × 34 across	–	one per wing per floor
Amenities	Lobby, fitness, loading	1200 / 900 / 600 GSF	–	ground floor only

5 How the layout is generated

This is the heart of the program. Each step is deterministic: the same inputs always give the same building.

5.1 Step 1: buildable envelope

With side setback s , front f , rear r , easement e on the east, and lot $W \times D$:

$$x_0 = s, \quad x_1 = W - \max(s, e), \quad y_0 = f, \quad y_1 = D - r.$$

The easement replaces the side setback on that side because it is wider. For the baseline: $x_0 = 10$, $x_1 = 200 - 20 = 180$, $y_0 = 15$, $y_1 = 300 - 20 = 280$, so the envelope is **170 ft $\times$ 265 ft = 45,050 sf**.

5.2 Step 2: how many of each unit

Studios and one-bedrooms are round($N \times \text{share}$); two-bedrooms get the remainder so the total is exactly N . For 120 units this gives 24 studios, 54 one-bedrooms and 42 two-bedrooms.

5.3 Step 3: spreading the unit types evenly

A proportional interleave decides the order of units along a corridor so studios, one- and two-bedrooms are mixed evenly rather than grouped. For step i of N , it picks the type with the largest gap between its target count $c_k \cdot i/N$ and the number already used. This is the same idea as largest-remainder apportionment.

5.4 Step 4: the shape of one wing

The bar's width is two rows of the deepest unit plus a corridor: $2 \times 36 + 6 = 78$ ft. A core (24 ft along the corridor, 34 ft across) sits at the start of each wing. Wings run along the *longer* side of the envelope (here the 265 ft depth). Several wings sit side by side with a 10 ft gap. A 24 ft driveway is placed on the west side if the wings plus 24 ft still fit across the envelope; otherwise the program tries a lane across the far end of the wings.

5.5 Step 5: storeys and heights

With surface parking, level 1 is 14 ft and upper levels 10.5 ft. With podium parking an extra 14 ft parking level sits below level 1, and residential levels are 10.5 ft. The building height is the sum of storey heights; the roof is a zero-height marker storey at the top.

5.6 Step 6: packing units floor by floor

Each wing has two rows (one each side of the corridor). Floor by floor, the program places units greedily:

1. Compute how many units this floor should take: $\lceil \text{units remaining} / \text{floors remaining} \rceil$, so units spread evenly.
2. On level 1, the lobby, fitness room and loading area (width = GSF/24 ft) take space at the start of the first row of wing 1, so fewer units fit on the ground floor.
3. For each next unit in the interleaved order, choose the row with the shortest used length (so rows balance), and place the unit if it fits.
4. Units are aligned to the corridor edge, so different depths line up along the hallway.

A wing's final length is the longest row length reached on any floor, so the wing is only as long as it needs to be. If units do not fit within the envelope length, the leftover units are reported as *not placed*.

5.7 Step 7: accessible units

The rule needs $\max(1, \lceil 0.02 N \rceil)$ accessible units. The first that many one-bedroom units are converted to ACCESSIBLE_1BR (3 units for 120, 2 for 60).

5.8 Step 8: parking

Required stalls = $\lceil \text{ratio} \times N \rceil$ (150 for 120 units at 1.25). For **surface** parking the program scans rows of 9×18 ft stalls, 42 ft apart (18 ft stall plus 24 ft aisle), inside the envelope, skipping any stall that touches a wing or the driveway, until the requirement is met or space runs out. For **podium** parking the capacity is estimated as $0.85 \times \text{footprint} / 330$ sf per stall (85 % usable, 330 sf per stall including aisles), and surface stalls only cover what podium cannot.

5.9 Step 9: choosing the number of wings

When wings is set to auto, the program builds designs with 1, 2 and 3 wings and keeps the best by a ranked score: (1) all units placed, (2) building inside the envelope, (3) a fire lane exists, (4) fewer wings.

6 The ten compliance checks

Each check gives PASS or FAIL so the reason for a failure is visible.

Check	Rule used
Units placed	Placed units $\geq$ requested units.
Stories	Storeys (including a podium level) $\leq$ zoning maximum.
Height	Sum of storey heights $\leq$ zoning maximum.
FAR	(Wing footprint $\times$ number of levels) / lot area $\leq$ zoning maximum.
Lot coverage	Total wing footprint / lot area $\leq$ zoning maximum %.
Parking stalls	Surface stalls + podium capacity $\geq$ required stalls.
Inside setbacks/easement	Every wing lies within the buildable envelope.
Fire lane	A 24 ft driveway exists (side or far end).
Accessible units	Accessible units $\geq \lceil 2\% \rceil$ of total.
Exit travel	Farthest unit distance from the core along the corridor $\leq$ 200 ft.

7 How the IFC file is written

Everything in the layout becomes a real IFC object. Lengths are converted from feet to metres (multiply by 0.3048), because IFC's default unit here is the metre.

Layout item	IFC entity	How
Project, site, building	IfcProject, IfcSite, IfcBuilding	Aggregated in a hierarchy.
Level / podium / roof	IfcBuildingStorey	Elevation set from storey heights.
Unit, core, amenity	$4 \times$ IfcWall	A box of four walls, 0.15 m thick, height = storey height.
Floor plates	IfcSlab	One per wing per level, 0.25 m thick, drawn from a polygon outline.
Roof	IfcRoof	Thin slab at the top level.
Ground, easement, drive, stalls	IfcSlab	Thin slabs at ground level for context.

Each object’s **name** encodes what it is, for example L2|STUDIO_A (level 2, studio unit) or SITE|stall. The viewer reads these names to pick colors and to filter by floor. Placement uses a 4×4 transform matrix (position plus rotation about the vertical axis); walls running north–south are rotated 90° . After writing, the file is opened again with IfcOpenShell to prove it is valid.

8 The viewer and the app

Reading the model. `read_meshes` reopens the `.ifc`, converts each element to triangles in world coordinates, and returns vertices and triangle indices. Because it reads the saved file, what you see is exactly what a BIM tool would open.

Drawing. Elements of the same kind (for example all studios) are merged into one Plotly `Mesh3d` trace, each with its own color: studio green, 1-bedroom blue, 2-bedroom orange, accessible purple, core maroon, amenities reds, slabs grey, easement yellow. You can drag to rotate, scroll to zoom, click a legend entry to hide a group, and use the “view floors up to” slider to peel the building back floor by floor.

The app. `streamlit run app.py` opens the tool in Chrome. The sidebar has scenario buttons (load a preset in one click), the request parser, and sliders for units, floors, mix, zoning, parking and wings. Any change regenerates the layout and IFC. Results are cached so repeat settings are instant. The top strip shows units, stories, FAR, coverage, parking and the number of failed checks. Tabs show the 3D model, a top-view site plan, the compliance table, and downloads for the `.ifc` and a standalone `.html` viewer.

9 Worked example: the 120-unit baseline

1. Lot $200 \times 300 = 60,000$ sf. Envelope $170 \times 265 = 45,050$ sf (Step 1).
2. 120 units become 24 studios, 54 one-bedrooms and 42 two-bedrooms (Step 2). Required parking $\lceil 1.25 \times 120 \rceil = 150$ stalls.
3. The program tries 1, 2 and 3 wings and keeps 3 wings (Step 9).
4. The wing footprint totals 47,463 sf, i.e. 79.1 % of the lot (limit 60 %) and FAR 3.16 (limit 2.5). Both fail.
5. Only 23 of 150 surface stalls fit in the free space, so parking fails.

The conclusion matches the earlier multi-agent study: 120 units with surface parking does not fit this site.

10 Results

Table 1: Scenario comparison, generated by this run (sample lot with 20 ft easement).

Scenario	Units	Stories	FAR	Cov. %	Parking	Checks failed
Baseline: 4 floors, surface parking	120/120	4	3.16	79.1	23/150	4
Right-sized: 60 units, podium (compliant)	60/60	5	2.31	46.3	75/75	0
Podium parking, 5 floors	120/120	6	3.74	62.4	120/150	7
Dense R-6, 7 floors + podium	120/120	8	4.01	50.2	90/90	3
Suburban R-2, 60 units, 3 floors	60/60	3	1.62	54.1	10/120	3

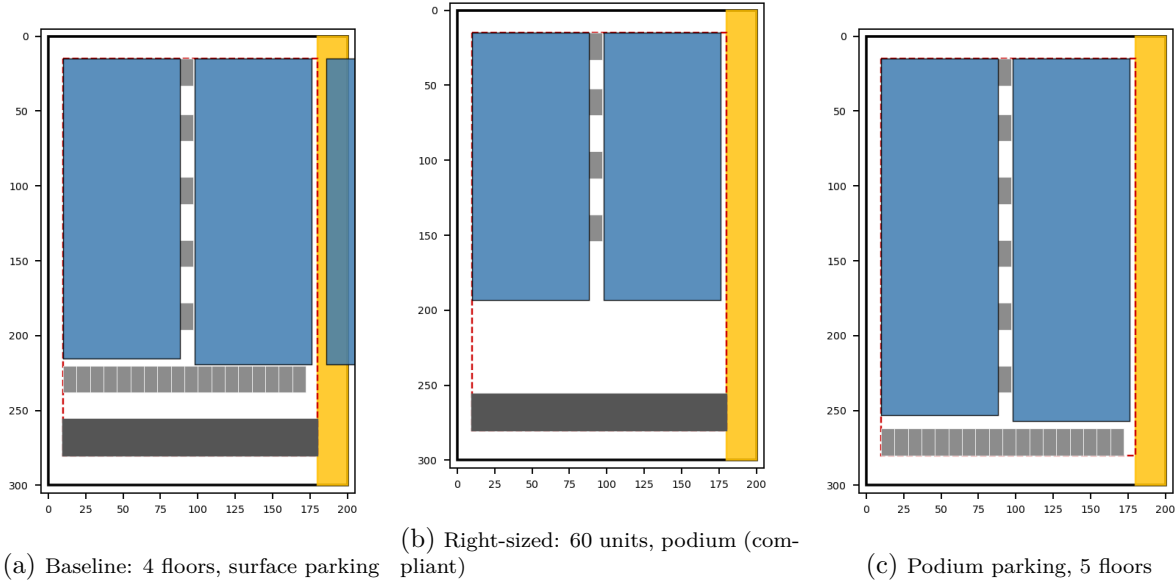


Figure 1: Top views (street at top; yellow = easement, dashed red = buildable envelope, grey = driveway and stalls, blue = building wings).



Figure 2: 3D renders read back from the IFC files (walls colored by unit type): baseline 120 units (left) and the right-sized 60-unit podium design (right).

10.1 Reading the scenarios

- **Baseline (120 units, 4 floors, surface):** the units all fit, but three wings need more width than the envelope offers, coverage and FAR exceed the R-4 limits, and only a fraction of the 150 stalls fit. This is a program/site mismatch, not a layout bug.
- **Right-sized (60 units, podium):** halving the program lets two wings fit inside the setbacks with a fire lane, and podium parking covers all 75 stalls. All ten checks pass.
- **Podium, 5 floors (120 units):** more height cuts the footprint, but the podium level makes 6 stories and pushes height past the limit.
- **Dense R-6:** looser coverage and parking rules help, but 7 floors plus a podium exceed the 7-story cap.
- **Suburban R-2:** strict FAR (1.0) and coverage (40 %) fail even at 60 units, showing how the zoning preset alone can decide feasibility.

Table 2: Compliance checks, right-sized 60-unit podium design.

Check	Required	Provided	Status
Units placed	60	60	PASS
Stories	≤ 5	5	PASS
Height (ft)	≤ 65	56.0	PASS
FAR	≤ 2.5	2.31	PASS
Lot coverage %	≤ 60	46.3	PASS
Parking stalls	75	75	PASS
Building inside setbacks/easement	yes	yes	PASS
Fire lane / driveway (24 ft)	yes	yes	PASS
Accessible units (2%)	2	2	PASS
Exit travel to core (ft)	≤ 200	154	PASS

Table 3: Compliance checks, 120-unit baseline.

Check	Required	Provided	Status
Units placed	120	120	PASS
Stories	≤ 5	4	PASS
Height (ft)	≤ 65	45.5	PASS
FAR	≤ 2.5	3.16	FAIL
Lot coverage %	≤ 60	79.1	FAIL
Parking stalls	150	23	FAIL
Building inside setbacks/easement	yes	no	FAIL
Fire lane / driveway (24 ft)	yes	yes	PASS
Accessible units (2%)	3	3	PASS
Exit travel to core (ft)	≤ 200	180	PASS

11 Testing

`test_design.py` contains 12 pytest tests, all passing:

- module counts always sum to the requested units;
- the right-sized scenario passes every check, and the baseline correctly fails parking;
- more floors shrink the footprint; the Suburban preset reports FAR ≤ 1.0 ;
- accessible units are created;
- for every scenario: the IFC has the right number of storeys and at least four walls per unit, and triangulation succeeds (round-trip);
- a headless Streamlit **AppTest** loads the app and clicks a scenario button.

One real bug was found in use: launching with `streamlit run app.py` from inside the folder crashed because `Path(__file__)` was a relative path. It was fixed by resolving the path, and the fix was checked by running the app with a relative path. Another problem found while writing this report was that early 3D renders showed only floors; walls-only rendering fixed it.

12 Limitations and next steps

- **Sample data.** Real jurisdiction rules and RNGD’s real BaaP catalog must replace the synthetic ones (the code needs only new data files).
- **Simple geometry.** Rectangular wings only; no L/U shapes, doors, windows, structure or MEP. Units are box walls.
- **Approximate parking.** A free-space scan with no full aisle-geometry check, and a rule-of-thumb podium capacity.
- **Proxy checks.** Fire lane, exit travel and accessibility are simplified stand-ins, not code determinations. A licensed professional must review any real design.
- **Next steps.** Add L/U wings, doors and windows, structural grid alignment, IFC property sets (areas, unit IDs) for quantity take-off, and connect the real BaaP data.

13 How to run

```
cd Testt/ifc_design
pip install ifcopenshell plotly
streamlit run app.py          # browser app (Chrome)
python generate.py            # writes out/*.ifc and out/*.html
python -m pytest test_design.py # 12 tests
python build_report.py        # rebuilds this PDF
```

Open any out/*.html by double-clicking; open any out/*.ifc in BIMvision, Blender-BIM, FreeCAD or the free Autodesk Viewer.